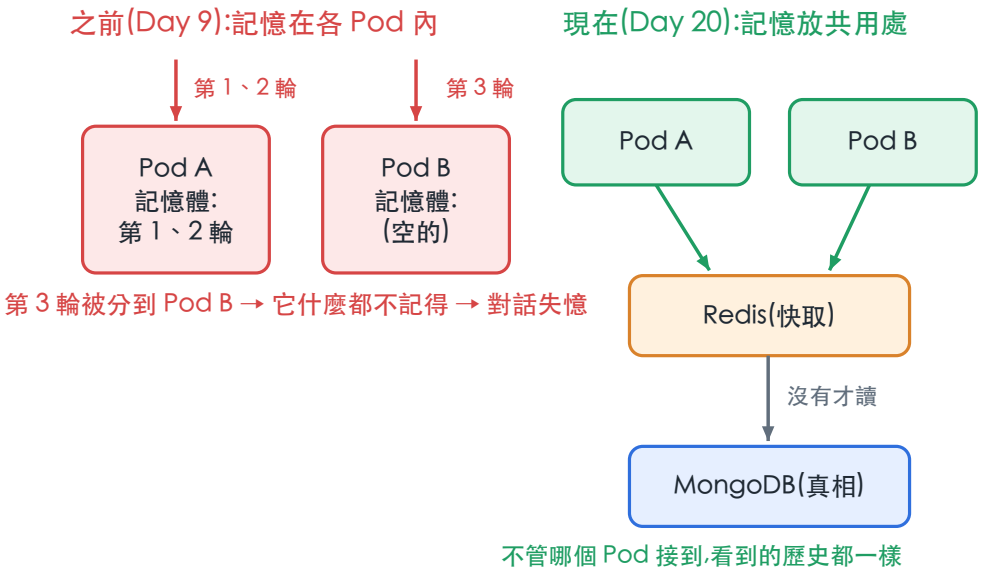
Day 20:Redis 快取

把「記憶放在 Pod 記憶體」改成「Mongo 真相 + Redis 快取」,真的修掉 Day 9 的 bug。附:今天踩到的三個坑、驗證數字,以及面試可以講的重點。

今天做的事	結果
寫 Redis cache-aside(chat_store.py)+ 7 個快取單元測試	21 個測試全過
Chart 新增 Redis Deployment / Service / 設定	ArgoCD 自動部署,Pod 正常
加 cache_requests_total 指標、Grafana 命中率面板	12 輪對話:10 hit / 2 miss
重跑 Day 9 多 replica 腳本	2 個 Pod 輪流回答,0 次記憶不一致
縮掉 Redis 再測	對話記憶仍正確,error 計數 +2,服務沒壞
加 MAX_OUTPUT_TOKENS=2048 限制輸出	成本控制的第二道(第一道是歷史長度上限)

1. 要修的是什麼 bug

Day 9 刻意把對話記憶放在每個 Pod 自己的記憶體。只有 1 個 Pod 時沒事;擴到 2 個以上,同一段對話被分到另一個 Pod 就失憶。這就是「無狀態服務」的意義:Pod 本身不能存狀態,狀態要放在共用的地方。

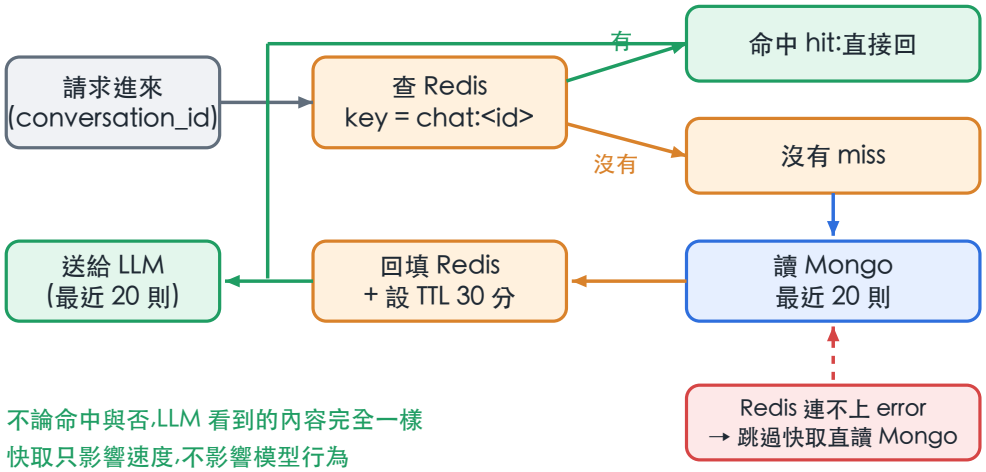


2. 為什麼是 Mongo + Redis 兩層

	MongoDB	Redis
角色	source of truth(真相)	快取(丟了可重建)
存放位置	硬碟(PVC)	記憶體
速度	較慢	很快
掛掉會怎樣	不能寫入 → 回 503(寧可失敗也不假裝成功)	降級直讀 Mongo,變慢但不壞
持久化	有	故意關掉(--save "" --appendonly no)

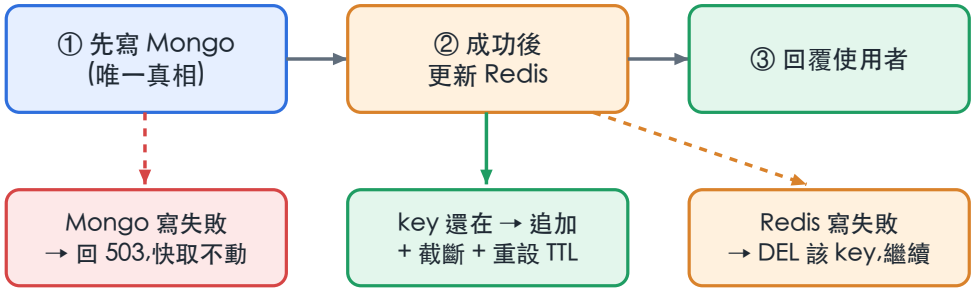
誠實講:這個規模只用 Mongo 就能修掉 Day 9 的 bug。Redis 的價值在「熱路徑延遲」和「減輕 DB 負載」,不是「沒有它就修不了」。面試時主動講這句,比硬說 Redis 必要更可信。

3. 讀取路徑(cache-aside)



不論命中與否,LLM 看到的內容完全一樣
快取只影響速度,不影響模型行為

4. 寫入路徑與失敗處理



key 不在就不寫:只塞 2 則會讓快取裡出現「殘缺的歷史」,
下次讀取從 Mongo 重建才正確。TTL 是最後一道保險。

情境	系統怎麼做	為什麼
Mongo 寫失敗	回 503,Redis 完全不動	避免「使用者看到回覆,但紀錄沒存」
Redis 寫失敗	嘗試 DEL 該 key,只記 log	下次讀取從 Mongo 重建;不能因快取壞掉讓服務壞掉
Redis 讀失敗 / 掛掉	跳過快取直讀 Mongo,記 error	降級而不是掛掉;Redis 逾時只設 0.5 秒,不拖慢請求
TTL 過期(30 分)	下次讀取 miss → Mongo 重建	最後一道保險,舊資料不會永遠留著
快取 key 已過期時寫入	不寫 Redis	只塞 2 則會變成殘缺歷史

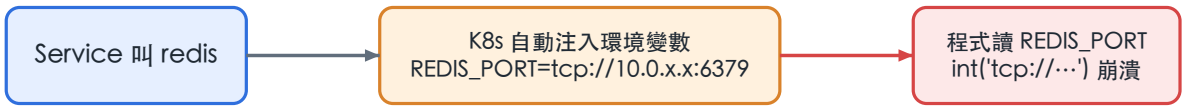
5. 驗證:真的修好了嗎

測試	結果
Day 9 腳本 12 輪,2 個 Pod 輪流回答	歷史每輪 +2(0,2,4⋯20),0 次不一致
快取指標	hit 10 / miss 2(新對話第一輪一定 miss)
Redis 縮到 0 再打 4 輪	記憶仍正確;error 計數 +2;沒有 5xx
單元測試	21 個:含 hit/miss、write-through、截斷、TTL 重建、Mongo 失敗、Redis 掛掉

小插曲:腳本在第 12 輪標了「不一致」,其實是腳本沒考慮「歷史上限 20 則」,預期值寫成 22。這是腳本的問題,不是系統的問題,已修正腳本。

6. 今天踩到的三個坑(面試好故事)

坑 ①:K8s 自動把 Service 變成環境變數

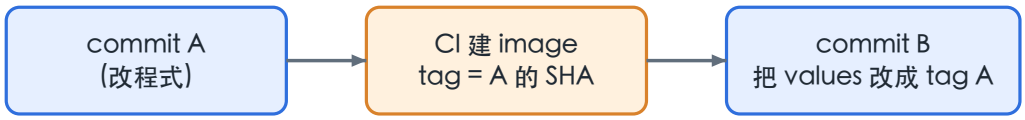


修法:Deployment 加 enableServiceLinks: false(用 DNS 名稱找服務,不需要這些變數)

ArgoCD 偵測到 git 變更 → 自動滾動更新 → 新 Pod 正常。全程沒下 kubectl apply。

教訓:K8s 預設會把同 namespace 的每個 Service 注入成環境變數(格式 <NAME>_PORT)。你自己的設定剛好同名就會被覆蓋。看 log 才找到:ValueError: invalid literal for int() ' tcp://10.0.186.231:6379'。

坑 ②:CI 的 image tag 先有雞還是先有蛋

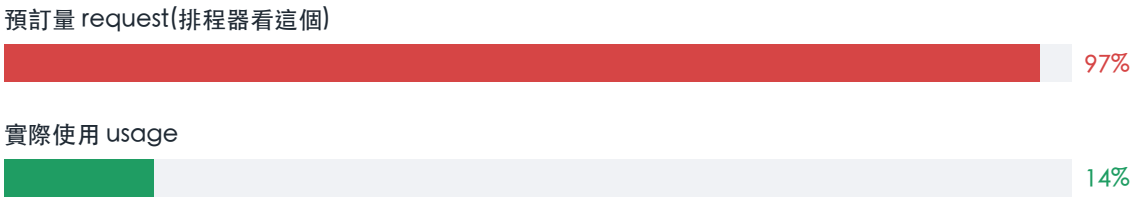


tag 必須是「已經建好」的 commit 的 SHA;改 tag 本身又是新 commit,
所以現在只能人工分兩次 push(A 要單獨先 push,CI 才會替它建 image)。

Day 22 要讓 CI 自動更新 tag,把這一步自動化。

坑 ③:CPU 預訂 97%,第 3 個 replica 排不進去

同一個節點(約 1900m CPU)兩種數字:



→ 機器很閒,但預訂已滿,所以第 3 個 backend Pod 排不進去(Pending)。

這是「預訂太浪費」,不是「CPU 不夠用」。

7. 面試可以問 / 可以講的點

Q:為什麼用 Redis?

對話歷史每次請求都要讀,走 Mongo 慢又吃 DB。Redis 放熱資料,命中就不碰 DB。但我會誠實說:這個規模只用 Mongo 也能修掉記憶 bug,Redis 的價值在延遲和 DB 負載,並拿得出 hit/miss 數據。

Q:Redis 和 Mongo 的資料不一致怎麼辦?(cache invalidation)

寫入順序固定:先寫 Mongo,成功才更新 Redis。Redis 寫失敗就 DEL 該 key;TTL 30 分鐘是最後保險。三層:寫入時更新、失敗時刪除、過期時重建。

Q:Redis 掛掉會怎樣?

降級為直讀 Mongo,變慢但不壞。我實測過:縮掉 Redis 後對話記憶仍正確,error 指標增加,沒有 5xx。Redis 逾時只設 0.5 秒,避免拖慢每個請求。

Q:什麼是 cache-aside?跟 write-through 差在哪?

cache-aside:程式自己負責查快取、miss 時讀 DB 並回填。我讀取用 cache-aside,寫入是「先寫 DB,再更新快取」(類似 write-through,但失敗時 DEL)。

Q:為什麼寫入時 key 不存在就不寫 Redis?

快取過期後若只追加 2 則,Redis 會有「殘缺的歷史」,下次命中就送給 LLM 錯誤的上下文。不寫的話下次 miss,從 Mongo 完整重建。

Q:快取會不會讓 LLM 看到不同內容?

不會。無論命中與否,送給 LLM 的永遠是最近 20 則。快取只影響延遲,不影響模型行為。我有單元測試驗證冷快取與熱快取的內容相同。

Q:你怎麼知道快取有效?

cache_requests_total{result=hit | miss | error} 接到 Grafana 看命中率。刻意把 error 與 miss 分開:miss 是正常現象,error 代表 Redis 壞了。

Q:最難除錯的一次?

Service 叫 redis,K8s 注入 REDIS_PORT=tcp://… 撞上我的設定,backend 一直重啟。看 kubectl logs 找到 ValueError;修法是 enableServiceLinks: false,然後 git push 讓 ArgoCD 自動修復。

Q:HPA 設 1~5,實際能到幾個?

實測只到 2 個。節點 CPU request 預訂 97%、實際使用只有 14%,排程看的是 request。要擴更多得降其他元件的預訂量或加節點。這是我主動做容量檢查才發現的。

8. 誠實的限制(README 要寫)

- Redis 單一實例、無 HA。掛掉時降級為直讀 Mongo,但沒有自動容錯。
- 同一段對話的併發請求沒有加鎖。極端情況下 Redis 與 Mongo 的順序可能暫時不一致,TTL 過期後自癒。
- 節點 CPU request 已近滿載。單一 B2ps_v2 節點,backend 實測最多 2 個 replica。
- 沒有真實負載測試。驗證都是功能面,沒有壓測數字。
- image tag 目前要人工更新。Day 22 會讓 CI 自動更新。
- Gemini 免費額度很小。所以多 replica 測試用假 LLM,沒有打真的模型。
- 對話內容會送到 LangSmith。隱私上要提醒使用者(Day 12 已列)。

接下來

- Day 21:緩衝日(rolling summary 為選做,可跳過)。
- Day 22–24:全流程整合——改 code → push → CI → ArgoCD 自動部署,連跑 3 次。要先在 GitLab 建一個專案 access token(write_repository)給 CI 自動更新 tag。
- Day 25–26:README、架構圖、已知限制、設計取捨。
- Day 27:故障演練(Gemini 逾時重試風暴、Redis 斷線降級)。